

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬ-
НОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ЛАБОРАТОРНАЯ 2

По дисциплине Контейнеризация и оркестрация приложений

Тема работы Знакомство с экосистемой Docker: от базовых команд к диагностике сервиса

Обучающийся Доценников Никита Андреевич

Факультет Прикладной информатики

Группа K3321

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся	_____	_____	Доценников Н.А.
	(дата)	(подпись)	(Ф.И.О.)
Руководитель	_____	_____	Аминов Н.С.
	(дата)	(подпись)	(Ф.И.О.)

Содержание

Персональные параметры	3
Этап 1. Первый рабочий образ	4
Задание 1.1	4
Задание 1.2	5
Задание 1.3	6
Задание 1.4	6
Этап 2. Оптимизация образа	8
Задание 2.1	8
Задание 2.2	8
Этап 3. <code>.dockerignore</code> — чистота в образе	10
Задание 3.1	10
Задание 3.2	10
Этап 4. Healthcheck — готовность к оркестрации	12
Задание 4.1	12
Задание 4.2	12
Задание 4.3	12
Этап 5. Финальный артефакт и тегирование	15
Задание 5.1	15
Задание 5.2	15
Приложение	17
<code>Dockerfile.optimized</code> :	17
<code>.dockerignore</code> :	18

Персональные параметры

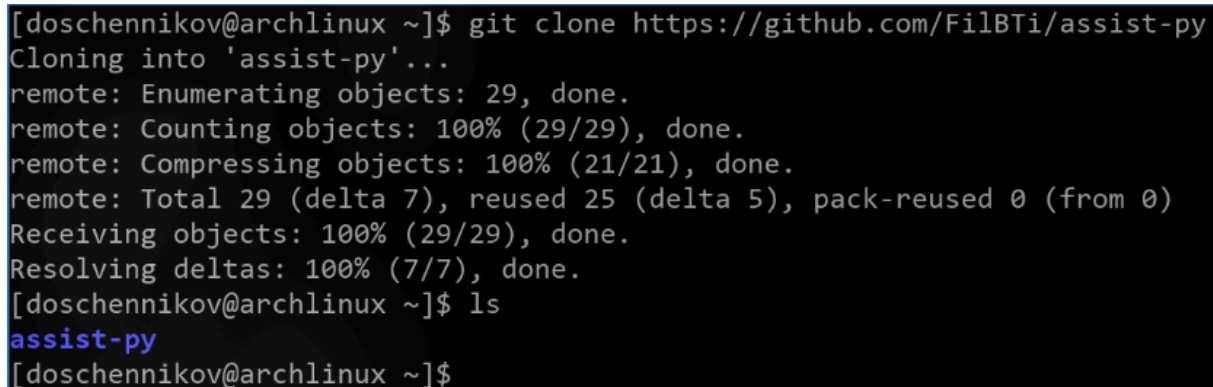
Параметр	Значение
Номер ИСУ	<i>465797</i>
Персональный порт	<i>5527</i>
Персональный префикс	<i>doschennikov</i>
Секретный ключ	<i>secret97doschennikov</i>

Таблица 1. Персональные параметры

Этап 1. Первый рабочий образ

Я клонировал репозиторий с приложением:

```
git clone https://github.com/FilBTi/assist-py
```



```
[doschennikov@archlinux ~]$ git clone https://github.com/FilBTi/assist-py
Cloning into 'assist-py'...
remote: Enumerating objects: 29, done.
remote: Counting objects: 100% (29/29), done.
remote: Compressing objects: 100% (21/21), done.
remote: Total 29 (delta 7), reused 25 (delta 5), pack-reused 0 (from 0)
Receiving objects: 100% (29/29), done.
Resolving deltas: 100% (7/7), done.
[doschennikov@archlinux ~]$ ls
assist-py
[doschennikov@archlinux ~]$
```

Рисунок 1. Клонирование репозитория

Задание 1.1

Я создал `Dockerfile` в корне проекта:

```
FROM python:3.11-slim

LABEL maintainer="doschennikov@lab.local"
LABEL version="1.0"

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ ./app/

EXPOSE 5527

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5527"]
```

```
[doschennikov@archlinux assist-py]$ ls
app Dockerfile README.md requirements.txt
[doschennikov@archlinux assist-py]$ cat Dockerfile
FROM python:3.11-slim

LABEL maintainer="doschennikov@lab.local"
LABEL version="1.0"

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ ./app/

EXPOSE 5527

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5527"]
[doschennikov@archlinux assist-py]$
```

Рисунок 2. Созданный Dockerfile

Задание 1.2

Я собрал образ:

```
docker build -t doschennikov-assistant:v1 .
```

```
pydantic-2.13.5 pydantic-core-2.46.5 python-dotenv-1.2.4 pyyaml-6.0.3 sniffio-1.3.1
starlette-0.38.6 typing-extensions-4.16.0 typing-inspection-0.4.4 uvicorn-0.30.6 uv
loop-0.23.0 watchfiles-1.3.0 websockets-17.1
WARNING: Running pip as the 'root' user can result in broken permissions and conflic
ting behaviour with the system package manager. It is recommended to use a virtual e
nvironment instead: https://pip.pypa.io/warnings/venv

[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
---> Removed intermediate container cc6bb288ad9f
---> 86466d1f0898
Step 7/9 : COPY app/ ./app/
---> d0ec0fbd4333
Step 8/9 : EXPOSE 5527
---> Running in cf40960b4794
---> Removed intermediate container cf40960b4794
---> 32f01ee8b462
Step 9/9 : CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5527"]
---> Running in b43dd1bfbb27
---> Removed intermediate container b43dd1bfbb27
---> 73501977b6d6
Successfully built 73501977b6d6
Successfully tagged doschennikov-assistant:v1
[doschennikov@archlinux assist-py]$
```

Рисунок 3. Собранный образ

Задание 1.3

Затем я запустил контейнер с переменными окружения:

```
docker run -d \
  --name doschennikov-assistant-1 \
  -p 5527:5527 \
  -e APP_OWNER=doschennikov \
  -e APP_SECRET=secret97doschennikov \
  doschennikov-assistant:v1
```

```
[doschennikov@archlinux assist-py]$ docker run -d --name doschennikov-assistant-1 -p 5527:5527 -e APP_OWNER=doschennikov -e APP_SECRET=secret97doschennikov doschennikov-assistant:v1
c0269f082ab46ce28c863a1cc9a866ea6a0aee2a4dc7d292e035d13187668763
[doschennikov@archlinux assist-py]$ _
```

Рисунок 4. Запущенный контейнер

Задание 1.4

Затем я проверил работоспособность:

- Проверка главной страницы:

```
curl http://localhost:5527/
```

```
[doschennikov@archlinux assist-py]$ curl http://localhost:5527/
{"message": "\u041f\u0440\u0438\u0432\u0435\u0442, doschennikov!", "service": "personal-assistant", "version": "1.0", "endpoints": ["/me", "/notes", "/weather/{city}", "/secret/{key}"]} [doschennikov@archlinux assist-py]$ _
```

Рисунок 5. Проверка главной страницы

- Проверка персонализации:

```
curl http://localhost:5527/me
```

```
[doschennikov@archlinux assist-py]$ curl http://localhost:5527/me
{"owner": "doschennikov", "secret_prefix": "secret"} [doschennikov@archlinux assist-py]$
```

Рисунок 6. Проверка персонализации

- Тест заметок:

```
curl -X POST http://localhost:5527/notes \
  -H "Content-Type: application/json" \
  -d '{"text": "\u041b\u041f2: \u0441\u0431\u043e\u0440\u043a\u0430 \u043e\u0431\u0440\u0430\u0437\u0430"}'

curl http://localhost:5527/notes
```

```
[doschennikov@archlinux assist-py]$ curl -X POST http://localhost:5527/notes \
> -H "Content-Type: application/json" \
> -d '{"text": "ЛР2: сборка образа"}'
{"status":"saved","note_id":0,"total":1}[doschennikov@archlinux assist-py]$ curl http://localhost:5527/notes
{"notes":["ЛР2: сборка образа"],"count":1}[doschennikov@archlinux assist-py]$ _
```

Рисунок 7. Тест заметок

- Тест секретного эндпоинта

```
curl http://localhost:5527/secret/secret97doschennikov
```

```
[doschennikov@archlinux ~]$ curl http://localhost:5527/secret/secret97doschennikov
{"flag":"LAB2_SUCCESS_DOSCHENNIKOV","message":"The secret key is correct!"}[doschennikov@archlinux ~]$ _
```

Рисунок 8. Тест секретного эндпоинта

Этап 2. Оптимизация образа

Задание 2.1

Я разбил Dockerfile на стадию сборки и получения финального образа:

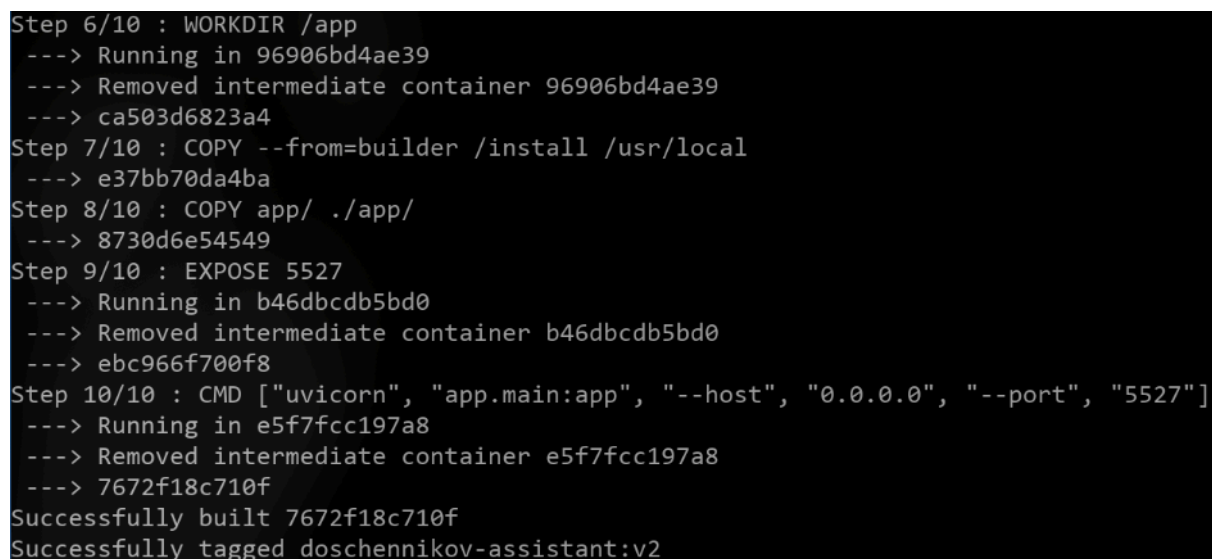
```
FROM python:3.11 AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /install /usr/local
COPY app/ ./app/
EXPOSE 5527
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5527"]
```

Задание 2.2

Я собрал оптимизированный образ:

```
docker build -t doschennikov-assistant:v2 -f Dockerfile.optimized .
```

A screenshot of a terminal window showing the output of a Docker build command. The output displays 10 steps: Step 6/10 (WORKDIR /app), Step 7/10 (COPY --from=builder /install /usr/local), Step 8/10 (COPY app/ ./app/), Step 9/10 (EXPOSE 5527), and Step 10/10 (CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5527"]). Each step shows the container ID being used and the removal of intermediate containers. The build concludes with 'Successfully built 7672f18c710f' and 'Successfully tagged doschennikov-assistant:v2'.

```
Step 6/10 : WORKDIR /app
---> Running in 96906bd4ae39
---> Removed intermediate container 96906bd4ae39
---> ca503d6823a4
Step 7/10 : COPY --from=builder /install /usr/local
---> e37bb70da4ba
Step 8/10 : COPY app/ ./app/
---> 8730d6e54549
Step 9/10 : EXPOSE 5527
---> Running in b46dbbdb5bd0
---> Removed intermediate container b46dbbdb5bd0
---> ebc966f700f8
Step 10/10 : CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5527"]
---> Running in e5f7fcc197a8
---> Removed intermediate container e5f7fcc197a8
---> 7672f18c710f
Successfully built 7672f18c710f
Successfully tagged doschennikov-assistant:v2
```

Рисунок 9. Результат сборки образа

И сравнил их размеры:

```
docker images | grep doschennikov-assistant
```

```
[doschennikov@archlinux assist-py]$ docker images | grep doschennikov-assistant
doschennikov-assistant:v1  73501977b6d6      255MB      62.1MB    U
doschennikov-assistant:v2  7672f18c710f      239MB      57.9MB
[doschennikov@archlinux assist-py]$
```

Рисунок 10. Результат сравнения размера образов

Размер образа уменьшился на 16MB (с 255MB до 239MB). Уменьшение даёт multi-stage сборка: зависимости устанавливаются в полном образе `python:3.11` (стадия `builder`), а в финальный образ на основе `python:3.11-slim` копируются только уже установленные пакеты, без инструментов сборки. Эффект небольшой, потому что первая версия образа уже была собрана на `slim`.

Копирование `requirements.txt` отдельно от кода ускоряет повторные сборки, потому что Docker кэширует каждый слой образа. Пока файл `requirements.txt` не изменился, слой с `pip install` берётся из кэша, и при правке бизнес-логики пересобирается только слой `COPY app/`. Если бы код копировался до установки зависимостей, то любое его изменение сбрасывало бы кэш, и все зависимости устанавливались бы заново.

Этап 3. `.dockerignore` — чистота в образе

Задание 3.1

В корне проекта я создал файл `.dockerignore` со следующим содержанием:

```
__pycache__/  
*.pyc  
*.pyo  
.git  
.gitignore  
.venv/  
venv/  
.env  
*.log  
Dockerfile*  
README.md
```

Задание 3.2

Я пересобрал образ командой:

```
docker build -t doschennikov-assistant:v3 .
```

```
Step 5/10 : FROM python:3.11-slim  
--> e41613d42d48  
Step 6/10 : WORKDIR /app  
--> Using cache  
--> ca503d6823a4  
Step 7/10 : COPY --from=builder /install /usr/local  
--> Using cache  
--> e37bb70da4ba  
Step 8/10 : COPY app/ ./app/  
--> Using cache  
--> 8730d6e54549  
Step 9/10 : EXPOSE 5527  
--> Using cache  
--> ebc966f700f8  
Step 10/10 : CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5527"]  
--> Using cache  
--> 7672f18c710f  
Successfully built 7672f18c710f  
Successfully tagged doschennikov-assistant:v3  
[doschennikov@archlinux assist-py]$ docker images | grep doschennikov-assistant  
doschennikov-assistant:v1  73501977b6d6      255MB      62.1MB     U  
doschennikov-assistant:v2  7672f18c710f      239MB      57.9MB  
doschennikov-assistant:v3  7672f18c710f      239MB      57.9MB  
[doschennikov@archlinux assist-py]$
```

Рисунок 11. Пересобранный образ

Без `.dockerignore` весь каталог проекта попадает в контекст сборки, и при инструкциях вроде `COPY . .`, а также при `COPY app/`, если файлы

лежат внутри `app/`, в образ могли бы попасть все файлы, которые сейчас перечислены в нём: кэш python (`__pycache__` , `*.pyc` , `*.pyo`), история Git (`.git` , `.gitignore`), локальные виртуальные окружения (`.venv/` , `venv/`), файл с переменными окружения `.env`, логи (`*.log`), а также `Dockerfile*` и `README.md`. Всё это увеличивает размер образа, ухудшает кэширование, так как любое изменение лишнего файла сбрасывает слой `COPY`, и может раскрыть лишнюю информацию.

Наличие директории `.git` в образе является потенциальной уязвимостью, так как в ней хранится вся история проекта, включая старые версии кода, адреса удалённых репозиторий, а также секреты (пароли, токены, ключи), которые когда-либо были закоммичены, даже если потом их удалили из текущей версии файлов. Любой, у кого есть доступ к образу, может запустить его, извлечь `.git` и восстановить эти данные командами `git log` и `git checkout`. Аналогично файл `.env` обычно содержит секреты в открытом виде.

Образ `v3` получил тот же идентификатор, что и `v2` (`7672f18c710f`), и все шаги были взяты из кэша (`Using cache`). Это ожидаемо, так как `.dockerignore` не затрагивает файлы, которые копируются в образ (`requirements.txt` и `app/`), поэтому содержимое слоёв не изменилось.

Этап 4. Healthcheck — готовность к оркестрации

Задание 4.1

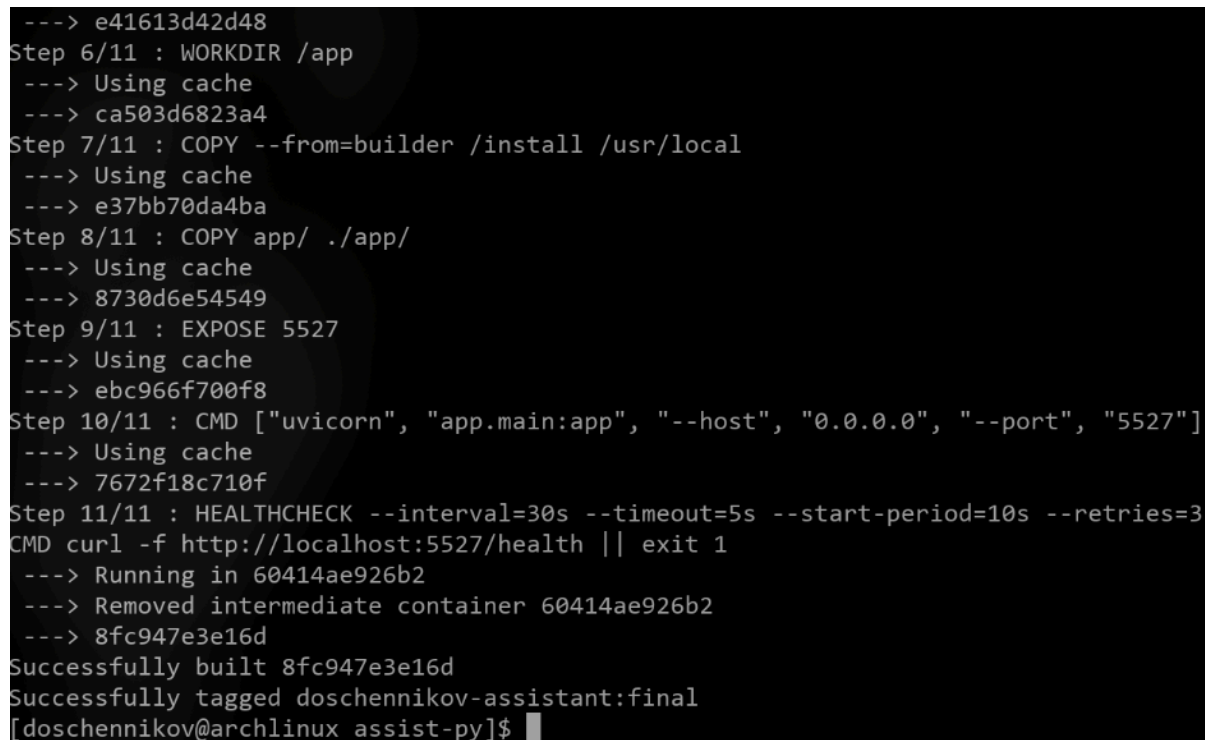
В конец `Dockerfile.optimized` я добавил:

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
\
CMD curl -f http://localhost:5527/health || exit 1
```

Задание 4.2

Я собрал финальную версию:

```
docker build -t doschennikov-assistant:final -f Dockerfile.optimized .
```



```
---> e41613d42d48
Step 6/11 : WORKDIR /app
---> Using cache
---> ca503d6823a4
Step 7/11 : COPY --from=builder /install /usr/local
---> Using cache
---> e37bb70da4ba
Step 8/11 : COPY app/ ./app/
---> Using cache
---> 8730d6e54549
Step 9/11 : EXPOSE 5527
---> Using cache
---> ebc966f700f8
Step 10/11 : CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5527"]
---> Using cache
---> 7672f18c710f
Step 11/11 : HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3
CMD curl -f http://localhost:5527/health || exit 1
---> Running in 60414ae926b2
---> Removed intermediate container 60414ae926b2
---> 8fc947e3e16d
Successfully built 8fc947e3e16d
Successfully tagged doschennikov-assistant:final
[doschennikov@archlinux assist-py]$
```

Рисунок 12. Сборка финальной версии

Задание 4.3

Я проверил статус здоровья. Запустил контейнер:

```
docker run -d \
--name doschennikov-assistant-final \
-p 5527:5527 \
-e APP_OWNER=doschennikov \
-e APP_SECRET=secret97doschennikov \
doschennikov-assistant:final
```

```
[doschennikov@archlinux assist-py]$ docker run -d \
> --name doschennikov-assistant-final \
> -p 5527:5527 \
> -e APP_OWNER=doschennikov \
> -e APP_SECRET=secret97doschennikov \
> doschennikov-assistant:final
553a6882be2c17ca44049ee694d07371e007b8c29860270d534dcb377cadedf30
[doschennikov@archlinux assist-py]$
```

Рисунок 13. Запуск контейнера

И проверил статус:

```
docker ps --filter "name=doschennikov-assistant-final" --format "table
{{.Names}}\t{{.Status}}"
```

После первого запуска контейнер получил статус `unhealthy`, хотя приложение работало и отвечало на запросы. Эндпоинт `/health` с хоста отвечал корректно:

```
curl http://localhost:5527/health
```

Ответ: `{"status":"healthy","owner":"doschennikov"}`. Я проверил, что делает проверка внутри контейнера:

```
docker exec doschennikov-assistant-final which curl
docker inspect --format='{{json .State.Health}}' doschennikov-
assistant-final
```

Команда `which curl` ничего не вернула, а в поле `Output` каждой проверки было `/bin/sh: 1: curl: not found`, при этом `FailingStreak` постоянно рос. Причина в том, что в образе `python:3.11-slim` нет утилиты `curl`.

Для исправления я добавил в финальную стадию `Dockerfile.optimized` установку утилиты `curl`:

```
RUN apt-get update && apt-get install -y --no-install-recommends curl
\
&& rm -rf /var/lib/apt/lists/*
```

После этого я пересобрал образ и запустил контейнер заново. Размер образа вырос с 239MB до 258MB из-за `curl`:

```
docker rm -f doschennikov-assistant-final
docker build -t doschennikov-assistant:final -f Dockerfile.optimized .
docker run -d \
  --name doschennikov-assistant-final \
  -p 5527:5527 \
```

```
-e APP_OWNER=doschennikov \  
-e APP_SECRET=secret97doschennikov \  
doschennikov-assistant:final  
sleep 10
```

После запуска статус контейнера стал **healthy** :

```
[doschennikov@archlinux assist-py]$ docker ps --filter "name=doschennikov-assistant-final" --format "table {{.Names}}\t{{.Status}}"  
NAMES          STATUS  
doschennikov-assistant-final  Up 4 minutes (healthy)  
[doschennikov@archlinux assist-py]$
```

Рисунок 14. Проверка статуса контейнера

Этап 5. Финальный артефакт и тегирование

Задание 5.1

Добавлен тег с версией:

```
docker tag doschennikov-assistant:final doschennikov-assistant:1.0.0
```

```
[doschennikov@archlinux assist-py]$ docker tag doschennikov-assistant:final doschennikov-assistant:1.0.0
[doschennikov@archlinux assist-py]$ docker images --format "table {{.Repository}}:{{.Tag}}\t{{.Size}}\t{{.ID}}" doschennikov-assistant
REPOSITORY:TAG          SIZE      IMAGE ID
doschennikov-assistant:1.0.0  258MB    a788967bd1af
doschennikov-assistant:final  258MB    a788967bd1af
doschennikov-assistant:v2    239MB    7672f18c710f
doschennikov-assistant:v3    239MB    7672f18c710f
doschennikov-assistant:v1    255MB    73501977b6d6
[doschennikov@archlinux assist-py]$
```

Рисунок 15. Тег с версией

Добавлен тег с датой сборки:

```
docker tag doschennikov-assistant:final doschennikov-assistant:$(date +%Y%m%d)
```

```
[doschennikov@archlinux assist-py]$ docker tag doschennikov-assistant:final doschennikov-assistant:$(date +%Y%m%d)
[doschennikov@archlinux assist-py]$ docker images --format "table {{.Repository}}:{{.Tag}}\t{{.Size}}\t{{.ID}}" doschennikov-assistant
REPOSITORY:TAG          SIZE      IMAGE ID
doschennikov-assistant:1.0.0  258MB    a788967bd1af
doschennikov-assistant:20261003  258MB    a788967bd1af
doschennikov-assistant:final  258MB    a788967bd1af
doschennikov-assistant:v2    239MB    7672f18c710f
doschennikov-assistant:v3    239MB    7672f18c710f
doschennikov-assistant:v1    255MB    73501977b6d6
[doschennikov@archlinux assist-py]$
```

Рисунок 16. Тег с датой сборки

Задание 5.2

Артефакт выполнения:

```
echo "===== " && \
echo "Сборка завершена: $(date '+%d.%m.%Y %H:%M:%S') " && \
echo "Студент: $(whoami)@$(hostname)" && \
echo "Образ: doschennikov-assistant" && \
echo "===== " && \
docker images | grep doschennikov-assistant | awk '{print $1, $2, $7}'
```

```

[doschennikov@archlinux assist-py]$ echo "====="
&& \
> echo "Сборка завершена: $(date '+%d.%m.%Y %H:%M:%S')" && \
> echo "Студент: $(whoami)@$(hostname)" && \
> echo "Образ: doschennikov-assistant" && \
> echo "=====" && \
> docker images | grep doschennikov-assistant | awk '{print $1, $2, $7}'
=====
Сборка завершена: 03.10.2026 12:25:39
Студент: doschennikov@archlinux
Образ: doschennikov-assistant
=====
doschennikov-assistant:1.0.0 a788967bd1af
doschennikov-assistant:20261003 a788967bd1af
doschennikov-assistant:final a788967bd1af
doschennikov-assistant:v1 73501977b6d6
doschennikov-assistant:v2 7672f18c710f
doschennikov-assistant:v3 7672f18c710f
[doschennikov@archlinux assist-py]$

```

Рисунок 17. Артефакт выполнения

Приложение

Dockerfile.optimized:

```
FROM python:3.11 AS builder

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.11-slim

LABEL maintainer="doschennikov@lab.local"
LABEL version="1.0"

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*

COPY --from=builder /install /usr/local

COPY app/ ./app/

EXPOSE 5527

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5527"]

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD curl -f http://localhost:5527/health || exit 1
```

.dockerignore :

```
__pycache__/  
*.pyc  
*.pyo  
.git  
.gitignore  
.venv/  
venv/  
.env  
*.log  
Dockerfile*  
README.md
```